ORIGINAL ARTICLE

The Bayesian Inference library for Python and R

Sebastian Sosa[1] | **Mary B. McElreath**[1] | **Cody T. Ross**[1]

[1]Department of Human Behavior, Ecology and Culture. Max Planck Institute for Evolutionary Anthropology. Leipzig, Germany

Correspondence
Sebastian Sosa
Email: sebastian_sosa@eva.mpg.de

Funding information
Max Planck Institute for Evolutionary Anthropology. Leipzig, Germany

1. Bayesian modeling is a powerful paradigm in modern statistics and machine learning. However, practitioners face significant obstacles in building bespoke models.
2. The landscape of Bayesian software is fragmented across programming languages and abstraction levels. Newcomers often gravitate towards high-level interfaces, like *R*, in order to use simple generalized linear models (*GLMs*) through interfaces like *brms*.
3. For niche problems, researchers must often transition to writing directly in lower-level programming languages, like *Stan* or *JAX*, which require specialist knowledge.
4. Furthermore, computational demands remain a significant bottleneck, often limiting the feasibility of applying Bayesian methods on large datasets and complex, high-dimensional models.
5. The Bayesian Inference (*BI*) is a cross-platform software distributed as a Python, R and Julia library. It provides an intuitive model-building syntax with the flexibility of low-level abstraction coding, while also providing pre-built *GLM* functions. Further, by facilitating hardware-accelerated GPU computation under-the-hood, *BI* permits high-dimensional models to be fit in a fraction of the time of comparable Stan models (up to 200-fold).

KEYWORDS
Bayesian methods, scalability, software, Python, R, Julia, GPU parallelization, social networks

1 | INTRODUCTION

Bayesian modeling has emerged as an essential part of the toolbox of modern statistics and machine learning, providing a framework for robust inference under uncertainty. Bayesian methods are valuable across the academic disciplines, but are especially useful in evolution, ecology, and animal behavior, where missing data, measurement bias, and non-standard (i.e., scientifically motivated) data-generating models are ubiquitous. The potential of Bayesian methods to address applied challenges in the field is large, but the current ecosystem of Bayesian software is difficult for many end-users (i.e., academic researchers) to navigate. Key challenges stem from the diverse array of different programming languages and packages users may be required to learn, trade-offs between ease-of-use and flexibility, and persistent computational scalability limitations for complex models or large datasets.

The first major obstacle is the fragmented landscape of Bayesian software, which features tools scattered across different programming languages (e.g., `Stan`, `TensorFlow probability (TFP)`, `NumPyro`, `JAX`) and software environments (e.g., `R`, `Python`, `Julia`). Within each of these settings, there are different notational conventions for defining the same types of models, and different packages require users to work at different levels of abstraction. Researchers may encounter domain-specific languages (like `Stan`), low level-of-abstraction libraries (like *PyMC*), or high level-of-abstraction libraries (like *brms* or *STRAND*, which aim to facilitate construction of general-use Stan models under-the-hood). This diverse landscape can prove difficult for new users, especially non-statisticians, to navigate. For instance, applied researchers new to Bayesian analysis may initially gravitate towards high level-of-abstraction solutions in their preferred programming environment (e.g., *brms* [1] in *R* [2]), only to find that their specific data analysis problem requires a bespoke solution that necessitates writing or editing raw Stan code. The Stan models, however, might be too slow to use, requiring users to rework their code into `JAX`, in order to permit GPU accelerated computation.

The difficulty of navigating such disparate software environments naturally leads practitioners to prioritize using tools that they are already familiar with, or that appear easiest to learn, even if they aren't necessarily the most appropriate tools for their research questions. As such, the diverse software landscape is characterized by an accessibility-flexibility trade-off. While high-level interfaces like *brms* offer an intuitive formula-based syntax for model definition, significantly lowering the initial barrier to entry—to the great benefit of the scientific community—this accessibility often comes at the cost of flexibility. Many generative scientific models are not easily defined in a formula-based syntax, and might require custom likelihood functions, multiple simultaneous equations, intricate prior structures, or other non-standard model components. For such models, the limitations of high-level wrappers become quite apparent. To gain the necessary flexibility, researchers must typically transition to lower-level probabilistic programming languages (PPLs), such as *Stan* [3], *PyMC*, *NumPyro*, or *TFP*. This transition imposes a much steeper learning curve, demanding a deeper understanding of probabilistic programming concepts (like computational graphs or tensor manipulation), and often requires users to write more verbose code. This significant jump in complexity can deter end-users, divert focus from statistical modeling to software engineering challenges, and ultimately slow down the pace of research.

Similar accessibility and flexibility constraints manifest as domain-specific limitations within specialized Bayesian packages. Fields like phylogenetics or network analysis benefit from tools such as *BEAST* [4], *RevBayes* [5], *STRAND* [6], or *BISON* [7], which provide accessible, pre-packaged models tailored to common domain-specific problems. A phylogeneticist might initially find *BEAST* convenient for running standard molecular clock models. However, when they wish to incorporate a novel evolutionary hypothesis requiring modification of the core model structure, or need to integrate data types not originally envisioned by the developers, they encounter rigid constraints. Extending these specialized tools frequently requires deep engagement with their underlying, often complex, codebase, or abandoning the domain-specific tool entirely in favor of writing bespoke code in a general-purpose PPL. This forces researchers to either compromise on their methodological innovations or undertake a significant software development effort.

Finally, even when model specification is achievable, with either general-purpose or specialized tools, computational feasibility remains a significant bottleneck for many Bayesian models. For the large datasets (e.g., millions of observations) and complex, high-dimensional models prevalent in modern research (e.g., in fields like genomics, neuroscience, and machine learning), Bayesian approaches can be prohibitively slow unless computation can be parallelized. While established tools like *Stan* feature highly optimized inference algorithms (particularly its NUTS sampler), high-dimension models can still take days-to-weeks to run (e.g., social network models in `STRAND` have a parameter complexity that scales with the square of the number of nodes, and so MCMC run-times can become unacceptable with as few as a few hundred individuals in the sample). Emerging frameworks built on *JAX* [8] (powering *NumPyro* and parts of *TFP*) promise substantial speedups via automatic differentiation, JIT compilation, and native support for parallel hardware architectures (e.g., GPUs and TPUs). Domain-specific tools often inherit the scalability limitations of the frameworks they are built upon, and there is now scope to consider building ease-of-use wrappers for JAX to complement those developed for Stan. This is an ongoing challenge.

Stepping back a bit, we contend that the added challenges of learning Bayesian methods, while simultaneously struggling with various programming language back-ends, may be discouraging end-users from fully exploring the capabilities of Bayesian methods, simply due to the friction introduced by language barriers. There is thus a pressing need for a Bayesian modeling package that synergistically addresses the interconnected limitations reviewed above. Here, we introduce *Bayesian Inference (BI)*, a software package designed to unify the modeling experience across the three dominant data-science languages, Python, R and Julia. *BI* tackles the *interoperability* barrier head-on by offering native interfaces in all environments. It aims to resolve the *accessibility-flexibility trade-off* by providing an intuitive model definition syntax reminiscent of `lm` in base R, while permitting advanced customization. *BI* also includes pre-built functions tailored for specialized models in areas like network analysis, survival analysis, and phylogenetic analysis, while still allowing extension and modification within its general framework. Crucially, *BI* enhances *scalability* by integrating with hardware-accelerated computation via *JAX* (using *NumPyro* or *TFP* as back-ends), which lets users choose between execution on CPUs, GPUs, and TPUs. By providing a streamlined, efficient, and unified environment for the end-to-end Bayesian workflow—from model specification and fitting, to diagnostics, and prediction—*BI* lowers the barrier-to-entry for sophisticated Bayesian analysis, allowing researchers across disciplines annd programming languages to confidently apply advanced Bayesian methods to their research problems.

2 | SOFTWARE PRESENTATION

2.1 | Interoperability

BI directly confronts the **interoperability** challenge by offering native, feature-equivalent implementations in Python, R and Julia. While minor syntactic differences exist to adhere to the idiomatic conventions of each language, the core model specification syntax, the procedural workflow for analysis, and the underlying computational engines remain fundamentally consistent. For instance, Python utilizes dot notation for method calls on class objects (e.g., `bi.dist.normal(0,1)`), while R employs dollar sign notation for accessing elements or methods within its object system (e.g., `bi$dist$normal(0,1)`). This triple-language availability significantly lowers the adoption barrier for researchers, allowing them to work entirely within their preferred programming environment without sacrificing access to a common, powerful Bayesian modeling framework. More critically, it supports seamless collaboration across heterogeneous programming environments, thereby enhancing reproducibility, methodological coherence, and interdisciplinary research.

2.2 | Accessibility

BI is designed to encapsulate the entire Bayesian modeling workflow within a cohesive object-oriented structure, promoting a streamlined and reproducible analysis pipeline. Typically, a user interacts with a primary `BI` object, through which they can sequentially:

1. **Handle Data:** Load, preprocess, and associate dataset(s) with the model object.
2. **Define Model:** Specify the model structure, including the likelihood(s), priors for all parameters, and incorporate any pre-built components using an intuitive formula syntax.
3. **Run Inference:** Execute the model fitting process using the No-U-Turn Sampler (NUTS), which triggers the back-end PPL (e.g., *NumPyro*, *TFP*) to perform Markov Chain Monte Carlo (MCMC) sampling. Progress indicators and diagnostics are typically provided.
4. **Analyze Posterior:** Access, summarize, and diagnose the posterior distributions of parameters. This includes methods for calculating posterior means, medians, credible intervals, convergence diagnostics (e.g., $\hat{R}$, Effective Sample Size - ESS), and retrieving raw posterior samples for custom analysis.
5. **Visualize Results:** Generate standard diagnostic plots (e.g., trace plots, rank plots, posterior distributions) and visualizations of model parameters, effects, and predictions using integrated plotting functions that leverage the *arviz* library.

This unified structure minimizes the need for users to juggle multiple disparate software tools or manually transfer data and results between different stages of the analysis, thereby enhancing efficiency and reproducibility. Finally, *BI* includes over 29 well-documented implementations of various standard and advanced Bayesian models Table 1. Examples include Generalized Linear Models (GLMs), Generalized Linear Mixed Models (GLMMs), survival analysis models (e.g., Cox proportional hazards), Principal Component Analysis (PCA), phylogenetic comparative methods, and various network models. Each implementation is accompanied by detailed documentation that encompasses: 1) general principles, 2) underlying assumptions, 3) code snippets in Python and R, and 4) mathematical details, enabling users to gain a deeper understanding of the modeling process and its nuances. Additionally, the framework's flexibility allows models to be combined; for example, building a zero-inflated model with varying intercepts and slopes, or constructing a joint model where principal components (derived from PCA) serve as predictors in a subsequent regression, allowing uncertainty to be propagated through all stages of the analysis.

2.3 | Accessibility-flexibility trade-off

BI is designed to navigate the critical *accessibility-flexibility trade-off* by providing multiple layers of abstraction and utility, catering effectively to users with varying levels of Bayesian modeling expertise and diverse complexity requirements through : simplified backend interaction via intuitive syntax, pre-built components for complex model features, addressing domain-specific limitations within a general framework, integrated End-to-End Workflow and extensive model library and documentation.

At its computational core, *BI* leverages the power and efficiency of established Probabilistic Programming Languages (PPLs) like *NumPyro* and *TFP*, both of which are built upon the *JAX* framework for high-performance numerical computation and automatic differentiation. However, *BI* deliberately abstracts away much of the inherent complexity of these lower-level tools (**Code block 1**). This significantly enhances **accessibility** for a broader range of users.

Code block 1: *Prior specification differences between NumPyro, TFP, and BI*

```
# NumPyro prior specification
numpyro.sample("mu", dist.Normal(0, 1)).expand([10])

# TFP prior specification (within a JointDistributionCoroutine)
yield Root(tfd.Sample(tfd.Normal(loc=1.0, scale=1.0), sample_shape=10))

# BI prior specification
bi.dist.normal(0, 1, name = "mu", shape = (10,))
```

To enhance **flexibility** without unduly sacrificing the accessibility provided by the high-level syntax, *BI* includes a library of pre-built, computationally optimized functions implemented directly in *JAX* (e.g., **Code block 2**). These components encapsulate common but potentially complex modeling structures, allowing users to incorporate them easily within the model specification. Key examples include:

1. *Varying effects* are implemented automatically, relieving the user from manually specifying the reparameterization. These cover hierarchical (multi-level) model components [9], including varying intercepts, varying slopes, and combined varying intercepts and slopes, with support for both *centered* and *non-centered* parameterizations. The non-centered parameterization—often essential for efficient sampling in hierarchical models, particularly when data are sparse—is handled internally.
2. *Kernels for Gaussian Processes* for modeling spatial, temporal, phylogenetic, or other forms of structured correlation or dependency.
3. *Gaussian mixture models* are implemented with automatic handling of component weights, means, and covariances, allowing flexible modeling of heterogeneous data. They provide a probabilistic framework for clustering and density estimation without requiring manual specification of component assignments.
4. *Dirichlet process mixture models* extend mixture modeling to an unbounded number of components, automatically adapting model complexity to the data. The nonparametric prior enables flexible clustering without the need to predefine the number of mixture components.
5. *Principal component analysis* is provided for dimensionality reduction, with automatic computation of orthogonal transformations that capture maximum variance. This allows efficient representation of high-dimensional data without manual specification of projection directions.
6. *Survival models* are supported with automatic handling of censored data, enabling inference on time-to-event outcomes. Parametric and semi-parametric formulations allow flexible modeling of hazard functions without requiring manual adjustment for incomplete observations.
7. *Block Model Effects* for implementing stochastic block models in network analysis.
8. *SRM effects* for modeling pairwise interactions in networks while accounting for sender effects, receiver effects, dyadic effects, nodal predictors, dyadic predictors, and observation biases [13].
9. *Network-Based Diffusion Approach (NBDA)* components for modeling the effect of network edges on the rates of transmission of phenomena (e.g., behavioral, epidemiological) while accounting for nodal or dyadic covariates [16].
10. *Network metrics* ranging from nodal, dyadic, and global network measures with a total of 11 that can be used to build custom models of social network analysis [15].
11. *Dense neural network layer* for modeling complex, non-linear relationships between variables [17].

These pre-built *JAX* functions provide tailored model components for common patterns in specific fields, while keeping them fully integrated within the general, extensible modeling framework (**Code block 2**). By providing these optimized building blocks within its general syntax, *BI* allows researchers in these fields to rapidly implement standard domain models using familiar concepts. Crucially, however, users retain the full flexibility of the *BI* framework to combine these domain-specific components with other model features (e.g., complex non-linear effects via splines, hierarchical structures across groups of networks or phylogenies) or to customize or extend them using *BI*'s underlying mechanisms if needed—a capability often missing in more narrowly focused domain-specific packages. This design aims to foster methodological innovation *within* specialized domains by lowering the barrier to implementing more complex or novel models.

Code block 2: *Random effect specification differences between NumPyro, TFP, and BI*

```
import numpyro.distributions as dist
import tensorflow_probability as tf
import tensorflow as tf
import jax.numpy as jnp
import jax.random as random
import numpyro

from BI import bi
tfd = tfp.distributions
m = bi()

# Numpyro version of random centered effect ----------------------------------------
a = numpyro.distributions.Normal(5, 2).sample(random.PRNGKey(0), sample_shape=(1,))
b = numpyro.distributions.Normal(-1, 0.5).sample(random.PRNGKey(0), sample_shape=(1,) )
sigma_cafe = numpyro.distributions.Exponential(1).sample(random.PRNGKey(0), sample_shape=(2,))
sigma = numpyro.distributions.Exponential(1).sample(random.PRNGKey(0))
Rho = numpyro.distributions.LKJ(2, 2).sample(random.PRNGKey(0))
cov = jnp.outer(sigma_cafe, sigma_cafe) * Rho
a_cafe_b_cafe =numpyro.distributions.MultivariateNormal(jnp.stack([a, b]), cov).sample(random.PRNGKey(0), sample_shape=(5,))
a_cafe, b_cafe = a_cafe_b_cafe[:, 0], a_cafe_b_cafe[:, 1]

# TFP version of random centered effect ----------------------------------------
a = tfd.Normal(5., 2.).sample()
b = tfd.Normal(-1., 0.5).sample()
sigma_cafe = tfd.Exponential(rate=1.).sample(sample_shape=(2,))
sigma = tfd.Exponential(rate=1.).sample()
Rho = tfd.LKJ(dimension=2, concentration=2.).sample()
cov = tf.einsum('i,j->ij', sigma_cafe, sigma_cafe) * Rho
a_cafe_b_cafe = tfd.MultivariateNormalFullCovariance(
  loc=tf.stack([a, b], axis=-1),  # shape (2,)
  covariance_matrix=cov
).sample(sample_shape=(5,))     # shape (5, 2)
a_cafe, b_cafe = a_cafe_b_cafe[:, 0], a_cafe_b_cafe[:, 1]

# BI version of random centered effect----------------------------------------
```

```
a = m.dist.normal(5, 2, name = 'a',sample=True)
b = m.dist.normal(-1, 0.5, name = 'b',sample=True)
sigma = m.dist.exponential( 1, name = 'sigma',sample=True)
varying_intercept, varying_slope = m.effects.varying_intercept_slope(
  N_group = 20,
  group = jnp.arrange(20), # dummy group
  global_intercept= a,
  global_slope= b,
  group_name = 'cafe',
  sample=True
)
```

2.4 | Scalability

Finally, a fundamental design principle of `BI` is to directly address the `scalability` challenge by building upon modern, high-performance computational backends. By utilizing *JAX*-based libraries like `NumPyro` and `TFP`, `BI` inherits their significant computational efficiencies, stemming from features such as Just-In-Time (JIT) compilation and seamless hardware acceleration (GPU/TPU support). These are critical for tackling the large datasets and complex models prevalent in contemporary research. This strategic reliance on the `JAX` ecosystem allows `BI` users to tackle computationally intensive problems that might be infeasible or prohibitively slow using frameworks lacking comparable optimization and hardware acceleration capabilities (e.g. NBDA for large networks). By abstracting the backend complexities while retaining their power, `BI` significantly enhances the practical `scalability` of sophisticated Bayesian inference for a wider audience.

3 | EXAMPLE : SRM MODEL

To illustrate how these design features of *BI* coalesce to provide a streamlined, flexible, and powerful solution, effectively addressing the limitations identified in the existing Bayesian software landscape we will provide a basic example of how an SRM model is declared in BI, compare it with the equivalent model in STAN (Appendix 1). We will also show how this model can be build from scratch with BI (**Code block 3**) or its custom functions (**Code block 4**) to highligh the aceessibility-flexibility of our package by demonstrating how advance user can build custom model (with less code than STAN) as well as how new user can apply pre-build *BI* models. Finally we show how it is also called in R (**Code block 5**) and Julia (**Code block 6**) to cross language use with *BI*. Readers interested in further details on data structure, data import, data manipulation, and model fitting for SRM models can refer directly to the *BI* documentation Modeling Network.

Code block 3: *SRM model from scratch with BI*

```
from BI import bi
m = bi()

def model(N_id, idx, result_outcomes,
     focal_individual_predictors,
     target_individual_predictors):

```

```
  # Intercept
  intercept = bi.dist.normal(
    logit(0.1/jnp.sqrt(N_id)),
    2.5, shape=(1,), name = 'intercept'
  )

  # Sender receiver ----------------------
  N_var = focal_individual_predictors.shape[0]
  N_id = focal_individual_predictors.shape[1]
  focal_effects = m.dist.normal(0, 1, name = 'focal_effects')
  target_effects = m.dist.normal( 0, 1, name = 'target_effects')
  terms = jnp.stack([
    focal_effects @ focal_individual_predictors,
    target_effects @ target_individual_predictors
    ], axis = -1)
  sr_raw = m.dist.normal(0, 1, shape=(2, N_id), name = 'sr_raw')
  sr_sigma = m.dist.exponential( 1, shape= (2,), name = 'sr_sigma')
  sr_L = m.dist.lkjcholesky(2, 2, name = "sr_L")
  rf = (((sr_L @ sr_raw).T * sr_sigma))
  ids = jnp.arange(0,sr_effects.shape[0])
  edgl_idx = m.net.vec_node_to_edgle(jnp.stack([ids, ids], axis = -1))
  sender = sr_effects[edgl_idx[:,0],0] + sr_effects[edgl_idx[:,1],1]
  receiver = sr_effects[edgl_idx[:,1],0] + sr_effects[edgl_idx[:,0],1]
  sr = jnp.stack([sender, receiver], axis = 1)

  # dyadic effects ------------------------------------------
  m.net.mat_to_edgl(dyadic_effect_mat)
  dr_raw = m.dist.normal(0, 1, shape=(2,N_dyads), name = 'dr_raw')
  dr_sigma = m.dist.exponential(1, name = 'dr_sigma' )
  dr_L = m.dist.lkjcholesky(2, 2, name = 'dr_L')
  dr_rf = deterministic('dr_rf', (
    ((dr_L @ dr_raw).T * jnp.repeat(dr_sigma, 2))
    ))

  dyad_effects = dist.normal(0, 1,
    name= 'dyad_effects', shape = (dyadic_predictors.ndim - 1,
  ))
  dr = dyad_effects * dyadic_predictors

  # Likelihood
  m.dist.poisson(jnp.exp(intercept + sr + dr), obs=result_outcomes)
```

Code block 4: *SRM model with prebuild functions in Python*

```
from BI import bi
m = bi()

def model(network, dyadic_predictors, focal_individual_predictors,
     target_individual_predictors,category):
    N_id = focal_individual_predictors.shape[0]
```

```
    # Block ---------------------------------------
    B_intercept = m.net.block_model(jnp.full((N_id,),0), 1, N_id, name = "B_intercept")

    ## SR shape =  N individuals---------------------------------------
    sr =  m.net.sender_receiver(
      focal_individual_predictors,
      target_individual_predictors,
      s_mu = 0.4, r_mu = -0.4
    )

    # Dyadic shape = N dyads--------------------------------------
    dr = m.net.dyadic_effect(dyadic_predictors, d_sd=2.5) # Diadic effect intercept only
    m.dist.bernoulli(logits = B_intercept + B_category + sr + dr, obs=network)
```

Code block 5: *SRM model with prebuild functions in R.*

```
library(BayesianInference)
m = imortBI()
model <- function(N_id, idxShape, result_outcomes,
        focal_individual_predictors, target_individual_predictors){

 x=0.1/jnp$sqrt(N_id)
 tmp=jnp$log(x / (1 - x))

 # Intercept
 intercept = bi.dist.normal(tmp, 2.5, shape=c(1), name = 'block')

 # SR
 sr = m$net$sender_receiver(
      focal_individual_predictors,
      target_individual_predictors
    )

 # Dyadic
 dr = m$net$dyadic_effect(shape = c(idxShape))

 # Likelihood
 bi.distpoisson(jnp$exp(intercept + sr + dr), obs=result_outcomes)
}
```

Code block 6: *SRM model with prebuild functions in Julia.*

```
using BayesianInference

# Setup device------------------------------------------------
m = importBI()
@BI function model(network, dyadic_predictors, focal_individual_predictors,
     target_individual_predictors,category)
```

```
    N_id = focal_individual_predictors.shape[0]

    # Block --------------------------------------
    B_intercept = m.net.block_model(jnp.full((N_id,),0), 1, N_id, name = "B_intercept")
    B_category = m.net.block_model(category, N_grp, N_by_grp, name = "B_category")

    ## SR shape =  N individuals---------------------------------------
    sr =  m.net.sender_receiver(
      focal_individual_predictors,
      target_individual_predictors,
      s_mu = 0.4, r_mu = -0.4
    )

    # Dyadic shape = N dyads--------------------------------------
    dr = m.net.dyadic_effect(dyadic_predictors, d_sd=2.5) # Diadic effect intercept only
    m.dist.bernoulli(logits = B_intercept + B_category + sr + dr, obs=network)
end
```

4 | SCALABILITY

Finally, we evaluate computational performance by comparing the execution time of the *Social Relations Model* (SRM) across networks of increasing size (50, 100, 200, and 400 nodes) using both *Stan* and *BI* (Figure 1a). For *BI*, computations were performed using both CPU and GPU implementations. CPU-based executions were carried out on a 120-core processor, while GPU-based executions were conducted on an NVIDIA A40 accelerator.

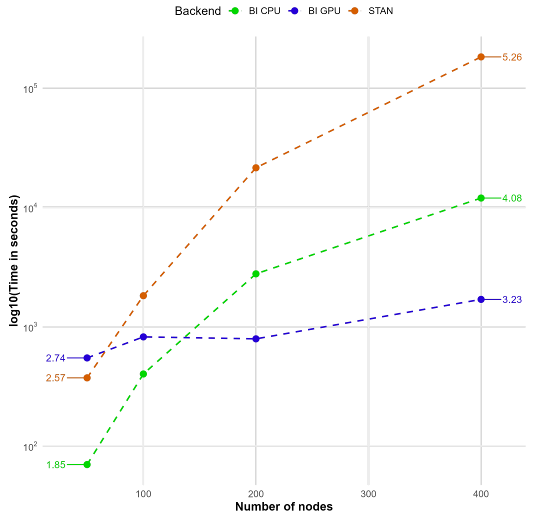
Overall, *BI* exhibits substantially faster computation times than *Stan*, with the performance gap widening markedly as network size increases. In particular, for networks with 400 nodes, *BI* achieves up to a 270-fold reduction in execution time relative to *Stan*. An additional noteworthy observation concerns GPU-based computation in *BI*: although GPU execution does not outperform CPU execution for smaller networks (e.g., 50 nodes), its rate of increase in computation time is considerably lower as network size grows, ultimately yielding superior performance for larger networks. Collectively, these results highlight the strong scalability of *BI* for complex models and large datasets, both on CPU and GPU architectures.

Finally, we demonstrate that parameter estimates obtained using *Stan* and *BI* for the SRM are highly consistent, as evidenced by the posterior distribution estimates for all networks analyzed in the benchmark tests Figure 1b. Code to reproduce the benchmark tests and the posterior comparisons are available in Appendix 2.

5 | DISCUSSION

BI framework is built on top of the popular Python programming language, with a focus on providing a user-friendly interface for model development and interpretation. Our framework is designed to be modular and extensible, allowing users to easily incorporate their own custom models and data types into the framework. One of the key features of this software is its comprehensive library of 27 predefined Bayesian models, covering a wide range of common applications and use cases. These models are accompanied by detailed explanations, making it easier for users to understand the underlying assumptions and apply the models to their specific research questions. In addition to these built-in models, the software includes several custom functions tailored for advanced statistical and network modeling.

(a)



(b)

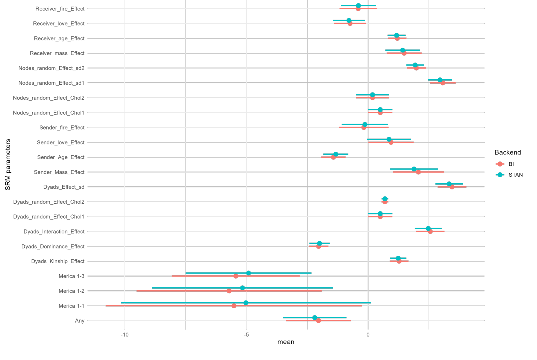


FIGURE 1 **(A)** Benchmarking runtime performance of STAN and BI (CPU and GPU implementations) for the Social Relations Model (SRM) across networks of varying sizes (50, 100, 200, and 400 nodes). **(B)** Posterior distributions for model parameters.

This curated library serves not only as a collection of ready-to-use tools but also as a valuable pedagogical resource, demonstrating best practices for constructing, fitting, and interpreting models within the *BI* framework, and providing robust templates for users aiming to develop novel model variants. Whether users are interested in hierarchical models, time-series analysis, or cutting-edge network modeling approaches, our library caters to a variety of analytical needs. This accessibility fosters an environment where users can confidently explore and implement Bayesian methods, ultimately enhancing their research capabilities.

By providing a streamlined and efficient environment for the end-to-end Bayesian workflow—from model specification and fitting to diagnostics and prediction, *BI* lowers the barrier to entry for sophisticated Bayesian modeling. We aim to empower a broader community of researchers across disciplines to confidently apply advanced Bayesian methods to their complex research problems; leveraging open-source, industry-standard frameworks such as NumPyro and TensorFlow Probability.

DATA AVAILABILITY STATEMENT

All implementations of *BI* are publicly available through their respective GitHub repositories for Python, R, and Julia. In addition, each implementation can be installed via the standard package repositories of the corresponding programming languages: pip for the Python version, CRAN for the R version, and the Julia package registry for the Julia version. Comprehensive documentation, including installation instructions, tutorials, and extended examples, are available on the official project website: https://s-sosa.com/BI.

Authors' Contributions

Sebastian Sosa conceived the research idea, developed the software, and co-authored the manuscript. Mary B. McElreath contributed to the writing of the manuscript. Cody T. Ross contributed to the development of the network models, the documentation of the models included in the software, and the writing of the manuscript.

FUNDING INFORMATION

This research has been funded by the Max Planck Institute for Evolutionary Anthropology, Deutscher Platz 6, 04103 Leipzig, Germany.

CONFLICT OF INTEREST STATEMENT

The authors declare that there are no conflicts of interest related to the content, data or conclusions presented in this study.

references

[1] Bürkner PC. brms: An R Package for Bayesian Multilevel Models Using Stan. Journal of Statistical Software 2017;80(1):1–28.

[2] Wickham H. R Packages. 1st ed. O'Reilly Media, Inc.; 2015.

[3] Stan Development Team, Stan Modeling Language Users Guide and Reference Manual, version 2.32;. `https://mc-stan.org`.

[4] Bouckaert R, Vaughan TG, Barido-Sottani J, Duchêne S, Fourment M, Gavryushkina A, et al. BEAST 2.5: An advanced software platform for Bayesian evolutionary analysis. PLOS Computational Biology 2019 04;15(4):1–28. https://doi.org/10.1371/journal.pcbi.1006650.

[5] Höhna S, Landis MJ, Heath TA, Boussau B, Lartillot N, Moore BR, et al. RevBayes: Bayesian Phylogenetic Inference Using Graphical Models and an Interactive Model-Specification Language. Systematic Biology 2016 05;65(4):726–736. https://doi.org/10.1093/sysbio/syw021.

[6] Ross CT, McElreath R, Redhead D. Modelling animal network data in R using STRAND. Journal of Animal Ecology 2024;93(3):254–266.

[7] Hart J, Weiss MN, Franks D, Brent L. BISoN: A Bayesian framework for inference of social networks. Methods in Ecology and Evolution 2023;14(9):2411–2420. https://besjournals.onlinelibrary.wiley.com/doi/abs/10.1111/2041-210X.14171.

[8] Bradbury J, Frostig R, Hawkins P, Johnson MJ, Leary C, Maclaurin D, et al., JAX: composable transformations of Python+NumPy programs; 2018. http://github.com/jax-ml/jax.

[9] McElreath R. Statistical rethinking: A Bayesian course with examples in R and Stan. Chapman and Hall/CRC; 2018.

[10] Tipping ME, Bishop CM. Probabilistic principal component analysis. Journal of the Royal Statistical Society Series B: Statistical Methodology 1999;61(3):611–622.

[11] Bishop CM, Nasrabadi NM. Pattern recognition and machine learning, vol. 4. Springer; 2006.

[12] Bingham E, Chen JP, Jankowiak M, Obermeyer F, Pradhan N, Karaletsos T, et al. Pyro: Deep Universal Probabilistic Programming. J Mach Learn Res 2019;20:28:1–28:6. http://jmlr.org/papers/v20/18-403.html.

[13] Sosa S, McElreath MB, Redhead D, Ross CT. Robust Bayesian analysis of animal networks subject to biases in sampling intensity and censoring. Methods in Ecology and Evolution;n/a(n/a). https://besjournals.onlinelibrary.wiley.com/doi/abs/10.1111/2041-210X.70017.

[14] Sosa S, Puga-Gonzalez I, Hu F, Pansanel J, Xie X, Sueur C. A multilevel statistical toolkit to study animal social networks: the Animal Network Toolkit Software (ANTs) R package. Scientific reports 2020;10(1):12507.

[15] Sosa S, Sueur C, Puga-Gonzalez I. Network measures in animal social network analysis: Their strengths, limits, interpretations and uses. Methods Ecol Evol 2021;12:10–21.

[16] Hoppitt W, Laland KN. Detecting social learning using networks: a users guide. American Journal of Primatology 2011;73(8):834–844.

[17] Phan D, Pradhan N, Jankowiak M. Composable Effects for Flexible and Accelerated Probabilistic Programming in NumPyro. arXiv preprint arXiv:191211554 2019;.

[18] Abril-Pla O, Andreani V, Carroll C, Dong L, Fonnesbeck CJ, Kochurov M, et al. PyMC: a modern, and comprehensive probabilistic programming framework in Python. PeerJ Computer Science 2023;9:e1516.

TABLE 1 Comprehensive list of all models available in BI

Documented Models	Source
Linear Regression	Statistical Rethinking, chapter 4 [9]
Multiple Regression	Statistical Rethinking, chapter 4 [9]
Interactions	Statistical Rethinking, chapter 8 [9]
Categorical models	Statistical Rethinking, chapter 5 [9]
Binomial models	Statistical Rethinking, chapter 11 [9]
Poisson models	Statistical Rethinking, chapter 11 [9]
Poisson models with offset	Statistical Rethinking, chapter 11 [9]
Gamma-Poisson models	Statistical Rethinking, chapter 11 [9]
Categorical models	Statistical Rethinking, chapter 11 [9]
Multinomial models	Statistical Rethinking, chapter 11 [9]
Dirichlet models	Statistical Rethinking, chapter X [9]
Beta-binomial models	Statistical Rethinking, chapter 12 [9]
Zero-inflated models	Statistical Rethinking, chapter 12 [9]
Varying-intercept models	Statistical Rethinking, chapter 13 [9]
Varying-intercept models	Statistical Rethinking, chapter 13 [9]
Gaussian process models	Statistical Rethinking, chapter 14 [9]
Measurement-error models	Statistical Rethinking, chapter 15 [9]
Missing-data models	Statistical Rethinking, chapter 15 [9]
Latent variable models	X [9]
Principal Component Analysis	Tipping & al. [10]
Gaussian Mixture Models	Bishop & al. [11]
Dirichlet Process Mixture Models	Pyro [12]
Basic social network models	STRAND [6]
Stochastic blockmodels	STRAND [6]
Network control for data collection biases	STRAND [13]
Network metrics	ANTs [14, 15]
Network-based diffusion analysis	NBDA [16]
Bayesian Neural Networks	Numpyro [17]
Survival Analysis	PYMC [18]